Tema 3

Utilización de objetos. Objetos nativos del lenguaje.



ÍNDICE

01 Utilización de objetos. Objetos nativos del lenguaje.

02 Interacción con el navegador. Objetos predefinidos asociados.

03 Generación de texto y elementos HTML desde código.

04 Gestión de la apariencia de la ventana.

05 Creación de nuevas ventanas. Comunicación entre ventanas.

06 Interacción con el usuario.

07 Mecanismos del navegador para el almacenamiento y recuperación de información.

Objetos nativos del lenguaje

- JavaScript cuenta con una colección de objetos predefinidos para trabajar con textos, números, fechas, listas y patrones de texto.
- El navegador añade otros para controlar la ventana, la dirección de la página, el historial, el almacenamiento o los permisos.
- **Manual:** MDN Web Docs

Objetos nativos del lenguaje: Date

- El objeto **Date** guarda un instante como los milisegundos transcurridos desde el 1 de enero de 1970 (UTC).
- Hay que tener en cuenta:
  - **Los meses empiezan en 0:** enero es el mes 0, pero los días empiezan por 1.
  - **Las fechas se desbordan en lugar de fallar:** el 32 de enero se convierte en el 1 de febrero.
  - **El formato del texto cambia la zona horaria:** **new Date("2026-10-30")** se interpreta en UTC, mientras que **new Date("2026-10-30T00:00")** se interpreta en la hora local.
  - **Son mutables:** **date.setDate(...)** modifica el objeto.

Objetos nativos del lenguaje: Date

➔ Algunos ejemplos:

Instrucción	**Dato obtenido**
getFullYear()	Año
getMonth()	Mes
getDate()	Día
getDay()	Día semana
getHours()	Hora
getMinutes()	Minutos
getSeconds()	Segundos
getMilliseconds()	Milisegundos
getTime()	Timestamp
getTimezoneOffset()	Timezone

Objetos nativos del lenguaje: String

- Al crear una cadena, esta se convierte de manera implícita en un objeto String, con sus propiedades y métodos predefinidos.
- Los métodos de manipulación de cadenas no modifican al objeto actual, sino que devuelven el objeto resultante de aplicar la modificación:
  - Cuando hacemos **name.toUpperCase()** sabemos que **name** no cambia.
  - Podemos guardar el resultado de la siguiente manera: **name = name.toUpperCase()**

Objetos nativos del lenguaje: String

➔ Algunos ejemplos:

Instrucción	**Dato obtenido**
toLowerCase() / toUpperCase()	Devuelve la cadena convertida a minúsculas/mayúsculas.
concat(cadena)	Devuelve el objeto con el valor de cadena concatenado al final.
charAt(posicion)	Devuelve el carácter que se encuentre en la posición solicitada.
indexOf(texto, [indice])	Devuelve la primera posición donde se encuentra el texto buscado, empezando a buscar desde la posición “indice”. Si “indice” no se indica, se toma por defecto el valor 0.
lastIndexOf (texto, [indice])	Busca “hacia atrás” la primera ocurrencia del texto buscado. Indice indica desde qué punto se empieza a buscar “hacia atrás”. Si no se indica el valor de “indice”, se busca desde el final. Si no existe “texto”, devuelve -1.
replace(texto1,texto2)	Busca ocurrencias de la cadena texto1 y las reemplaza por texto2.
split(caracter, [trozos])	Separa la cadena mediante un carácter separador. Trozos indica el máximo de separaciones. Si no se indica, se harán todas las separaciones posibles.
substring(inicio, [fin])	Devuelve la subcadena comprendida entre la posición inicio y la posición fin. Si fin no se indica, se toma como valor el final de la cadena.

Objetos nativos del lenguaje: Math

- Contiene una colección de métodos y constantes.
- Algunas constantes:

Constante	**Valor**
E	Número de Euler
PI	Número Pi
LN2 / LN10	Logaritmo neperiano de 2 / 10
LOG2E / LOG10E	Logaritmo en base 2 / 10 del número de Euler

Objetos nativos del lenguaje: Math

➔ Algunos métodos

Método	**Acción**
floor(numero)	Redondea hacia abajo un número
ceil(numero)	Redondea hacia arriba un número
round(numero)	Redondea al entero más cercano
abs(numero)	Devuelve el número en valor absoluto
max / min(x,y)	Devuelve el mayor / menor de dos números x e y
pow(x,y)	Devuelve x elevado a y
random()	Devuelve un número aleatorio con decimales entre 0 y 1
sqrt(numero)	Devuelve la raíz cuadrada del número indicado

Interacción con el navegador

➔ Algunos objetos:

Objeto	**Función**
location	La dirección actual. Cambiarla para navegar.
document	El documento HTML (DOM).
console	Mostrar información de depuración.
history	El historial de la pestaña actual.
navigator	Información del navegador y acceso a APIs.
screen	La pantalla del dispositivo.
window (tamaño)	El área visible de la pantalla.
alert / confirm / prompt	Diálogos del sistema. Bloquean la página.
localStorage	Almacenamiento persistente en el navegador.
sessionStorage	Almacenamiento durante la sesión.
fetch	Peticiones HTTP.

Interacción con el navegador: URL

➔ Para trabajar con direcciones (o algunas de sus partes) no es necesario partir las cadenas a mano.

- **new URL(texto)** separa el protocolo, el dominio, la ruta y los parámetros
- **URLSearchParams** lee y modifica los parámetros de consulta codificándolos correctamente.

```
const url = new URL("https://tienda.es/buscar?q=teclado mecánico&page=2");
url.hostname;                          // "tienda.es"
url.searchParams.get("q");             // "teclado mecánico"
url.searchParams.set("page", "3");
url.toString();                        // "https://tienda.es/buscar?q=teclado+mec%C3%A1nico&page=3"

// Cambiar la URL de la página sin recargar (como las SPA)
history.pushState({ pagina: 3 }, "", url.pathname + url.search);
window.addEventListener("popstate", (e) => mostrarPagina(e.state?.pagina ?? 1));   // botón atrás
```

Generación de texto y elementos HTML desde código

➔ En JavaScript, la generación de texto y elementos HTML puede hacerse de tres formas:

- **document.write**: la más antigua y ahora mismo desaconsejada.
- **innerHTML**: interpreta un texto como HTML. Peligroso con datos de usuario.
- **document.createElement** y **elemento.textContent**: la forma segura.

```
<!doctype html>
<html>
  <head>
  </head>
  <body>
    <div id="contenedor">
    </div>
    <script>
      let contenedor = document.querySelector("#contenedor")
      contenedor.textContent = "Esto se escribe desde Javascript"
    </script>
  </body>
</html>
```

Gestión de la apariencia de la ventana

➔ Usando **window** se puede modificar, entre otros parámetros, el tamaño de una ventana.

- **win.resizeBy(width,height):** Cambiar el tamaño de la ventana según el width/height dado en relación con el tamaño actual. Se permiten valores negativos.
- **win.resizeTo(width,height):** Redimensionar la ventana al tamaño dado.
- También existe el evento **window.onresize.**

Creación de nuevas ventanas. Comunicación entre ventanas.

- **window.open(url, nombre, opciones)**: abre una ventana o pestaña nueva y devuelve una referencia a ella.
- Los navegadores bloquean las ventanas emergentes que cuyo origen no es un clic del usuario.
- Las ventanas de distinto origen no pueden acceder la una al documento de la otra.
  - Usan **postMessage** para comunicarse y quien recibe el mensaje debe comprobar siempre de dónde viene.

Creación de nuevas ventanas. Comunicación entre ventanas.

➔ Un ejemplo:

```
<script>
    let ventana = window.open(
        "https://jocarsa.com",
        "miVentana",
        "width=800,height=600,left=200,top=100"
    );
</script>
```

Interacción con el usuario

➔ Es posible capturar acciones del teclado o ratón por parte del usuario:

- Ratón:
  - **onclick**: captura un clic simple.
  - **ondblclick**: captura un doble clic.
  - **onmousemove**: captura el movimiento.
- Teclado:
  - **onfocus**: se ha seleccionado el cuadro de introducción de texto.
  - **onblur**: se ha deseleccionado el cuadro de introducción de texto.
  - **keydown**: se ha pulsado una tecla.

Mecanismos del navegador para el almacenamiento y recuperación de información

- Las cookies son pequeños textos que el navegador guarda para un sitio web y envía al servidor en cada petición.
- Se leen y se escriben con **document.cookie**.

```
<script>
  // Guardar
  document.cookie = "nombre=Jose";

  // Leer
  console.log(document.cookie);
</script>
```

Mecanismos del navegador para el almacenamiento y recuperación de información

➔ Para guardar datos que solo necesita el navegador existen:

- **localStorage**: persiste aunque se cierre el navegador.
- **sessionStorage**: se borra al cerrar la pestaña.
- **IndexedDB**: para volúmenes grandes o datos estructurados.

Mecanismos del navegador para el almacenamiento y recuperación de información

➔ Ejemplo **localStorage**:

```
<script>
  // Guardar
  localStorage.setItem("nombre", "Jose");

  // Leer
  console.log(localStorage.getItem("nombre"));
</script>
```

Mecanismos del navegador para el almacenamiento y recuperación de información

➔ Ejemplo **sessionStorage**:

```
<script>
  // Guardar
  sessionStorage.setItem("nombre", "Jose");

  // Leer
  console.log(sessionStorage.getItem("nombre"));
</script>
```